

Punto 4.5 - Solución a N+1: Include, proyecciones y Split Queries

Práctica

Ejercicio: Añadir métodos al repositorio del proyecto AceriaData que apliquen las soluciones al problema N+1: Include, ThenInclude, proyecciones y AsSplitQuery. Comparar el número de consultas y el tiempo de ejecución de cada técnica. Analizar el SQL generado.

Contexto del proyecto: En el punto 4.4 se identificó el problema N+1, observando su impacto en el rendimiento y analizando el SQL generado. En este punto se profundiza en las soluciones: Include, ThenInclude, proyecciones y AsSplitQuery. Estas soluciones se usarán en el punto 4.6 para el over-fetching y en el punto 4.8 para las Split Queries.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de solución al problema N+1 a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerConPlanchasInclude();
    List<OrdenFabricacion> ObtenerConAleacionesThenInclude();
    List<OrdenFabricacion> ObtenerConPlanchasYDetalleSplitQuery();
    List<OrdenConPlanchasYDetalleDto> ObtenerResumenConPlanchasYDetalleProyeccion();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerConPlanchasInclude(); → método que usa Include para cargar las planchas.

Línea 11: List ObtenerConAleacionesThenInclude(); → método que usa ThenInclude para cargar las aleaciones.

Línea 12: List ObtenerConPlanchasYDetalleSplitQuery(); → método que usa AsSplitQuery para cargar planchas y detalle.

Línea 13: `List ObtenerResumenConPlanchasYDetalleProyeccion();` → método que usa proyección para cargar un resumen con planchas y detalle.

Línea 15: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 16: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

| Paso 3: Implementar los métodos de solución en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs`:

```

public List<OrdenFabricacion> ObtenerConPlanchasInclude()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Planchas)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerConAleacionesThenInclude()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerConPlanchasYDetalleSplitQuery()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .AsSplitQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenConPlanchasYDetalleDto> ObtenerResumenConPlanchasYDetalleProyeccion()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConPlanchasYDetalleDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            Planchas = o.Planchas.Select(p => new PlanchaDto
            {
                Id = p.Id,
                Espesor = p.Espesor,
                Ancho = p.Ancho,
                Largo = p.Largo,
                Peso = p.Peso,
                Activa = p.Activa
            }).ToList(),
            Detalle = o.Detalle == null ? null : new DetalleDto
            {
                ComposicionQuimica = o.Detalle.ComposicionQuimica,
                TemperaturaColada = o.Detalle.TemperaturaColada,
                Notas = o.Detalle.Notas
            }
        })
}
    
```

```

    })
    .ToList();
}

```

Línea 1: `public List ObtenerConPlanchasInclude()` → declara el método.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 5: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 6: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 7: `.ToList();` → materializa la consulta.

Línea 10: `public List ObtenerConAleacionesThenInclude()` → declara el método.

Línea 12: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 13: `.AsNoTrackingWithIdentityResolution()` → aplica resolución de identidad.

Línea 14: `.Include(o => o.OrdenesAleaciones)` → incluye la colección de entidades intermedias.

Línea 15: `.ThenInclude(oa => oa.Aleacion)` → incluye la aleación de cada entidad intermedia.

Línea 16: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 17: `.ToList();` → materializa la consulta.

Línea 20: `public List ObtenerConPlanchasYDetalleSplitQuery()` → declara el método.

Línea 22: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 23: `.AsNoTrackingWithIdentityResolution()` → aplica resolución de identidad.

Línea 24: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 25: `.Include(o => o.Detalle)` → incluye la referencia al detalle.

Línea 26: `.AsSplitQuery()` → divide la consulta en varias.

Línea 27: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 28: `.ToList();` → materializa la consulta.

Línea 31: `public List ObtenerResumenConPlanchasYDetalleProyeccion()` → declara el método.

Línea 33: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 34: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 35: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 36: `.Select(o => new OrdenConPlanchasYDetalleDto` → proyecta al DTO.

Línea 37: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 38: `Cliente = o.Cliente,` → asigna el cliente.

Línea 39: `Estado = o.Estado,` → asigna el estado.

Línea 40: `Planchas = o.Planchas.Select(p => new PlanchaDto` → proyecta las planchas.

Línea 41: `Id = p.Id,` → asigna el Id.

Línea 42: `Espesor = p.Espesor,` → asigna el espesor.

Línea 43: `Ancho = p.Ancho,` → asigna el ancho.

Línea 44: `Largo = p.Largo,` → asigna el largo.

Línea 45: `Peso = p.Peso`, → asigna el peso.

Línea 46: `Activa = p.Activa` → asigna el estado activo.

Línea 47: `}).ToList()`, → materializa la colección.

Línea 48: `Detalle = o.Detalle == null ? null : new DetalleDto` → comprueba si el detalle existe.

Línea 49: `ComposicionQuimica = o.Detalle.ComposicionQuimica`, → asigna la composición química.

Línea 50: `TemperaturaColada = o.Detalle.TemperaturaColada`, → asigna la temperatura.

Línea 51: `Notas = o.Detalle.Notas` → asigna las notas.

Línea 52: `}` → cierra la proyección del detalle.

Línea 53: `})` → cierra la proyección.

Línea 54: `.ToList()`; → materializa la consulta.

Error común: si se usa `Include` con dos colecciones sin `AsSplitQuery`, EF Core genera un producto cartesiano. Se debe usar `AsSplitQuery` en esos casos.

| Paso 4: Crear el caso de uso de solución al problema N+1

Crear el archivo `src/AceriaData.Application/UseCases/SolucionN1UseCase.cs`:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class SolucionN1UseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public SolucionN1UseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== SOLUCIÓN AL PROBLEMA N+1 ===");

        DemostrarConInclude();
        DemostrarConThenInclude();
        DemostrarConSplitQuery();
        DemostrarConProyeccion();
        CompararTodasLasSoluciones();
    }

    private void DemostrarConInclude()
    {
        Console.WriteLine("\n--- Solución con Include ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerConPlanchasInclude();
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($"  {orden.NumeroOrden}: {orden.Planchas.Count} planchas");
        }
    }

    private void DemostrarConThenInclude()
    {
        Console.WriteLine("\n--- Solución con ThenInclude ---");
    }
}
```

```

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerConAleacionesThenInclude();
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($"  {orden.NumeroOrden}: {orden.OrdenesAleaciones.Count} aleaciones");
        }
    }

    private void DemostrarConSplitQuery()
    {
        Console.WriteLine("\n--- Solución con AsSplitQuery ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerConPlanchasYDetalleSplitQuery();
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var orden in ordenes)
        {
            var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica;
            Console.WriteLine($"  {orden.NumeroOrden}: {orden.Planchas.Count} planchas | Detalle:
{detalle}");
        }
    }

    private void DemostrarConProyeccion()
    {
        Console.WriteLine("\n--- Solución con proyección ---");

        var cronometro = Stopwatch.StartNew();
        var resúmenes = _unidad.Ordenes.ObtenerResumenConPlanchasYDetalleProyeccion();
        cronometro.Stop();

        Console.WriteLine($"Resúmenes: {resúmenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds}
ms");
        foreach (var resumen in resúmenes)
        {
            var detalle = resumen.Detalle == null ? "Sin detalle" : resumen.Detalle.ComposicionQuimica;
            Console.WriteLine($"  {resumen.NumeroOrden}: {resumen.Planchas.Count} planchas | Detalle:
{detalle}");
        }
    }

    private void CompararTodasLasSoluciones()
    {
        Console.WriteLine("\n--- Comparación de todas las soluciones ---");

        var cronometroInclude = Stopwatch.StartNew();
        var ordenesInclude = _unidad.Ordenes.ObtenerConPlanchasInclude();
        cronometroInclude.Stop();

        var cronometroThenInclude = Stopwatch.StartNew();
        var ordenesThenInclude = _unidad.Ordenes.ObtenerConAleacionesThenInclude();
        cronometroThenInclude.Stop();

        var cronometroSplitQuery = Stopwatch.StartNew();
        var ordenesSplitQuery = _unidad.Ordenes.ObtenerConPlanchasYDetalleSplitQuery();
        cronometroSplitQuery.Stop();

        var cronometroProyeccion = Stopwatch.StartNew();
        var resúmenesProyeccion = _unidad.Ordenes.ObtenerResumenConPlanchasYDetalleProyeccion();
        cronometroProyeccion.Stop();

        Console.WriteLine($"Include: {cronometroInclude.ElapsedMilliseconds} ms");
        Console.WriteLine($"ThenInclude: {cronometroThenInclude.ElapsedMilliseconds} ms");
        Console.WriteLine($"AsSplitQuery: {cronometroSplitQuery.ElapsedMilliseconds} ms");
        Console.WriteLine($"Proyección: {cronometroProyeccion.ElapsedMilliseconds} ms");
    }
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class SolucionN1UseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public SolucionN1UseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== SOLUCIÓN AL PROBLEMA N+1 ==="); → muestra la cabecera.

Línea 19: DemostrarConInclude(); → llama al método de Include.

Línea 20: DemostrarConThenInclude(); → llama al método de ThenInclude.

Línea 21: DemostrarConSplitQuery(); → llama al método de AsSplitQuery.

Línea 22: DemostrarConProyeccion(); → llama al método de proyección.

Línea 23: CompararTodasLasSoluciones(); → llama al método de comparación.

Línea 26: private void DemostrarConInclude() → declara el método.

Línea 28: Console.WriteLine("\n--- Solución con Include ---"); → muestra la cabecera.

Línea 30: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 31: var ordenes = _unidad.Ordenes.ObtenerConPlanchasInclude(); → carga las órdenes con Include.

Línea 32: cronometro.Stop(); → detiene el cronómetro.

Línea 34: Console.WriteLine(\$"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms"); → muestra el número de órdenes y el tiempo.

Línea 35: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 37: Console.WriteLine(\$" {orden.NumeroOrden}: {orden.Planchas.Count} planchas"); → muestra el número de planchas.

Línea 40: private void DemostrarConThenInclude() → declara el método.

Línea 42: Console.WriteLine("\n--- Solución con ThenInclude ---"); → muestra la cabecera.

Línea 44: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 45: var ordenes = _unidad.Ordenes.ObtenerConAleacionesThenInclude(); → carga las órdenes con ThenInclude.

Línea 46: cronometro.Stop(); → detiene el cronómetro.

Línea 48: Console.WriteLine(\$"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms"); → muestra el número de órdenes y el tiempo.

Línea 49: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 51: Console.WriteLine(\$" {orden.NumeroOrden}: {orden.OrdenesAleaciones.Count} aleaciones"); → muestra el número de aleaciones.

Línea 54: private void DemostrarConSplitQuery() → declara el método.

Línea 56: Console.WriteLine("\n--- Solución con AsSplitQuery ---"); → muestra la cabecera.

Línea 58: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 59: `var ordenes = _unidad Ordenes.ObtenerConPlanchasYDetalleSplitQuery();` → carga las órdenes con `AsSplitQuery`.

Línea 60: `cronometro.Stop();` → detiene el cronómetro.

Línea 62: `Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el número de órdenes y el tiempo.

Línea 63: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 65: `var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica;` → obtiene el detalle.

Línea 66: `Console.WriteLine($" {orden.NumeroOrden}: {orden.Planchas.Count} planchas | Detalle: {detalle}");` → muestra los datos.

Línea 69: `private void DemostrarConProyeccion()` → declara el método.

Línea 71: `Console.WriteLine("\n--- Solución con proyección ---");` → muestra la cabecera.

Línea 73: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 74: `var resúmenes = _unidad Ordenes.ObtenerResumenConPlanchasYDetalleProyeccion();` → carga los resúmenes con proyección.

Línea 75: `cronometro.Stop();` → detiene el cronómetro.

Línea 77: `Console.WriteLine($"Resúmenes: {resúmenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el número de resúmenes y el tiempo.

Línea 78: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 80: `var detalle = resumen.Detalle == null ? "Sin detalle" : resumen.Detalle.ComposicionQuimica;` → obtiene el detalle.

Línea 81: `Console.WriteLine($" {resumen.NumeroOrden}: {resumen.Planchas.Count} planchas | Detalle: {detalle}");` → muestra los datos.

Línea 85: `private void CompararTodasLasSoluciones()` → declara el método.

Línea 87: `Console.WriteLine("\n--- Comparación de todas las soluciones ---");` → muestra la cabecera.

Línea 89: `var cronometroInclude = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 90: `var ordenesInclude = _unidad Ordenes.ObtenerConPlanchasInclude();` → carga las órdenes con `Include`.

Línea 91: `cronometroInclude.Stop();` → detiene el cronómetro.

Línea 93: `var cronometroThenInclude = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 94: `var ordenesThenInclude = _unidad Ordenes.ObtenerConAleacionesThenInclude();` → carga las órdenes con `ThenInclude`.

Línea 95: `cronometroThenInclude.Stop();` → detiene el cronómetro.

Línea 97: `var cronometroSplitQuery = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 98: `var ordenesSplitQuery = _unidad Ordenes.ObtenerConPlanchasYDetalleSplitQuery();` → carga las órdenes con `AsSplitQuery`.

Línea 99: `cronometroSplitQuery.Stop();` → detiene el cronómetro.

Línea 101: `var cronometroProyeccion = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 102: `var resúmenesProyeccion = _unidad Ordenes.ObtenerResumenConPlanchasYDetalleProyeccion();` → carga los resúmenes con proyección.

Línea 103: `cronometroProyeccion.Stop();` → detiene el cronómetro.

Línea 105: `Console.WriteLine($"Include: {cronometroInclude.ElapsedMilliseconds} ms");` → muestra el tiempo de Include.

Línea 106: `Console.WriteLine($"ThenInclude: {cronometroThenInclude.ElapsedMilliseconds} ms");` → muestra el tiempo de ThenInclude.

Línea 107: `Console.WriteLine($"AsSplitQuery: {cronometroSplitQuery.ElapsedMilliseconds} ms");` → muestra el tiempo de AsSplitQuery.

Línea 108: `Console.WriteLine($"Proyección: {cronometroProyeccion.ElapsedMilliseconds} ms");` → muestra el tiempo de proyección.

Error común: si se ejecutan las consultas en el mismo DbContext, las entidades de las consultas anteriores permanecen en el Change Tracker. Se debe usar un DbContext distinto para cada consulta si se quiere medir el tiempo de forma aislada.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<SolucionN1UseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<SolucionN1UseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 7: Insertar datos de prueba con planchas, detalle y aleaciones

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes, planchas, detalle y aleaciones:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var aleacion1 = new Aleacion { Nombre = "AISI 1045",Codigo = "A1045", PorcentajeCarbono = 0.45m,
    PorcentajeManganeso = 0.75m, Activo = true };
    var aleacion2 = new Aleacion { Nombre = "AISI 4140",Codigo = "A4140", PorcentajeCarbono = 0.40m,
    PorcentajeManganeso = 0.85m, Activo = true };
    context.Aleaciones.AddRange(aleacion1, aleacion2);
    context.SaveChanges();
}
```

```

var ordenes = new List<OrdenFabricacion>
{
    new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
    new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) },
    new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) },
    new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Oeste", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) },
    new OrdenFabricacion { NumeroOrden = "OF-2024-0005", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 5, 12) },
    new OrdenFabricacion { NumeroOrden = "OF-2024-0006", Cliente = "Constructora del Sur", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 6, 18) }
};
context.OrdenesFabricacion.AddRange(ordenes);
context.SaveChanges();

var planchas = new List<PlanchaAcero>
{
    new PlanchaAcero { OrdenId = ordenes[0].Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true },
    new PlanchaAcero { OrdenId = ordenes[0].Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true },
    new PlanchaAcero { OrdenId = ordenes[1].Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true },
    new PlanchaAcero { OrdenId = ordenes[2].Id, Espesor = 15.0, Ancho = 1800, Largo = 3500, Peso = 450.0m, Activa = true },
    new PlanchaAcero { OrdenId = ordenes[3].Id, Espesor = 9.0, Ancho = 1100, Largo = 2200, Peso = 180.0m, Activa = true },
    new PlanchaAcero { OrdenId = ordenes[4].Id, Espesor = 11.0, Ancho = 1300, Largo = 2700, Peso = 290.0m, Activa = true }
};
context.PlanchasAcero.AddRange(planchas);

var detalle1 = new DetalleOrden { OrdenId = ordenes[0].Id, ComposicionQuimica = "C: 0.45%, Mn: 0.75%", TemperaturaColada = 1550.5 };
context.DetallesOrden.Add(detalle1);

var ordenAleacion1 = new OrdenAleacion { OrdenFabricacionId = ordenes[0].Id, AleacionId = aleacion1.Id, CantidadUtilizada = 1500.5m, EstadoRelacion = "Activa" };
var ordenAleacion2 = new OrdenAleacion { OrdenFabricacionId = ordenes[0].Id, AleacionId = aleacion2.Id, CantidadUtilizada = 800.0m, EstadoRelacion = "Activa" };
var ordenAleacion3 = new OrdenAleacion { OrdenFabricacionId = ordenes[1].Id, AleacionId = aleacion1.Id, CantidadUtilizada = 1200.0m, EstadoRelacion = "Activa" };
context.OrdenesAleaciones.AddRange(ordenAleacion1, ordenAleacion2, ordenAleacion3);

context.SaveChanges();
}

```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: var aleacion1 = new Aleacion { ... }; → crea la primera aleación.

Línea 8: var aleacion2 = new Aleacion { ... }; → crea la segunda aleación.

Línea 9: context.Aleaciones.AddRange(aleacion1, aleacion2); → registra las aleaciones.

Línea 10: context.SaveChanges(); → inserta las aleaciones.

Línea 12: var ordenes = new List → crea la lista de órdenes.

Línea 14: new OrdenFabricacion { ... }, → primera orden.

Línea 15: new OrdenFabricacion { ... }, → segunda orden.

Línea 16: `new OrdenFabricacion { ... },` → tercera orden.

Línea 17: `new OrdenFabricacion { ... },` → cuarta orden.

Línea 18: `new OrdenFabricacion { ... },` → quinta orden.

Línea 19: `new OrdenFabricacion { ... }` → sexta orden.

Línea 20: `};` → cierra la lista.

Línea 21: `context.OrdenesFabricacion.AddRange(ordenes);` → registra las órdenes.

Línea 22: `context.SaveChanges();` → inserta las órdenes.

Línea 24: `var planchas = new List` → crea la lista de planchas.

Línea 26: `new PlanchaAcero { ... },` → primera plancha.

Línea 27: `new PlanchaAcero { ... },` → segunda plancha.

Línea 28: `new PlanchaAcero { ... },` → tercera plancha.

Línea 29: `new PlanchaAcero { ... },` → cuarta plancha.

Línea 30: `new PlanchaAcero { ... },` → quinta plancha.

Línea 31: `new PlanchaAcero { ... }` → sexta plancha.

Línea 32: `};` → cierra la lista.

Línea 33: `context.PlanchasAcero.AddRange(planchas);` → registra las planchas.

Línea 35: `var detalle1 = new DetalleOrden { ... };` → crea el detalle.

Línea 36: `context.DetallesOrden.Add(detalle1);` → registra el detalle.

Línea 38: `var ordenAleacion1 = new OrdenAleacion { ... };` → crea la primera relación.

Línea 39: `var ordenAleacion2 = new OrdenAleacion { ... };` → crea la segunda relación.

Línea 40: `var ordenAleacion3 = new OrdenAleacion { ... };` → crea la tercera relación.

Línea 41: `context.OrdenesAleaciones.AddRange(ordenAleacion1, ordenAleacion2, ordenAleacion3);` → registra las relaciones.

Línea 43: `context.SaveChanges();` → inserta las planchas, el detalle y las relaciones.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

| Paso 8: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada solución. Se observa el tiempo de cada técnica y el número de consultas ejecutadas en el log.

| Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```

=== SOLUCIÓN AL PROBLEMA N+1 ===

--- Solución con Include ---
Órdenes: 6 | Tiempo: 45 ms
  OF-2024-0001: 2 planchas
  OF-2024-0002: 1 planchas
  OF-2024-0003: 1 planchas
  OF-2024-0004: 1 planchas
  OF-2024-0005: 1 planchas
  OF-2024-0006: 0 planchas

--- Solución con ThenInclude ---
Órdenes: 6 | Tiempo: 48 ms
  OF-2024-0001: 2 aleaciones
  OF-2024-0002: 1 aleaciones
  OF-2024-0003: 0 aleaciones
  OF-2024-0004: 0 aleaciones
  OF-2024-0005: 0 aleaciones
  OF-2024-0006: 0 aleaciones

--- Solución con AsSplitQuery ---
Órdenes: 6 | Tiempo: 52 ms
  OF-2024-0001: 2 planchas | Detalle: C: 0.45%, Mn: 0.75%
  OF-2024-0002: 1 planchas | Detalle: Sin detalle
  ...

--- Solución con proyección ---
Resúmenes: 6 | Tiempo: 40 ms
  OF-2024-0001: 2 planchas | Detalle: C: 0.45%, Mn: 0.75%
  OF-2024-0002: 1 planchas | Detalle: Sin detalle
  ...

--- Comparación de todas las soluciones ---
Include: 45 ms
ThenInclude: 48 ms
AsSplitQuery: 52 ms
Proyección: 40 ms

```

La primera sección muestra el tiempo de Include. La segunda sección muestra el tiempo de ThenInclude. La tercera sección muestra el tiempo de AsSplitQuery. La cuarta sección muestra el tiempo de proyección. La quinta sección muestra la comparación.

Observaciones: la proyección es la técnica más rápida porque carga menos datos. AsSplitQuery es ligeramente más lenta porque ejecuta varias consultas. Include y ThenInclude tienen tiempos similares. La diferencia entre las técnicas es pequeña porque la tabla tiene pocas filas. En tablas con muchas filas, las diferencias son mayores.

| Paso 10: Diagnosticar un error común

Modificar el método ObtenerConPlanchasYDetalleSplitQuery para eliminar AsSplitQuery:

```

public List<OrdenFabricacion> ObtenerConPlanchasYDetalleSplitQuery()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

```

Resultado esperado: el código funciona, pero la consulta genera un producto cartesiano porque se incluyen dos colecciones. Cada plancha se combina con cada detalle. El número de filas del resultado es mayor que el número de órdenes.

Solución: añadir AsSplitQuery para dividir la consulta en varias.

```
public List<OrdenFabricacion> ObtenerConPlanchasYDetalleSplitQuery()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .AsSplitQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Resultado esperado con la solución: la consulta se divide en varias y no se produce el producto cartesiano.

Errores comunes del ejercicio

Error	Causa	Solución
N+1	No se usó Include	Usar Include
Producto cartesiano	Se incluyeron dos colecciones sin AsSplitQuery	Usar AsSplitQuery
Proyección sin ToList	Se olvidó materializar la colección	Añadir ToList dentro de la proyección
Tiempo alto	Se cargan todas las columnas	Usar proyección
Segundo nivel sin ThenInclude	Se accede a una propiedad de segundo nivel	Usar ThenInclude
Resolución de identidad no usada	Se cargan entidades relacionadas duplicadas	Usar AsNoTrackingWithIdentityResolution

Reto resuelto: Combinar Include, ThenInclude y AsSplitQuery

Reto: Crear un método en el repositorio que cargue las órdenes con sus planchas, su detalle y sus aleaciones en una sola operación. Usar Include, ThenInclude, AsNoTrackingWithIdentityResolution y AsSplitQuery. Comparar el tiempo con el problema N+1.

Solución paso a paso:

Paso 1: Añadir el método a la interfaz IOrdenRepositorio:

```
List<OrdenFabricacion> ObtenerOrdenesCompletasConSplitQuery();
```

Línea 1: List ObtenerOrdenesCompletasConSplitQuery(); → declara el método.

Paso 2: Implementar el método en OrdenRepositorio:

```
public List<OrdenFabricacion> ObtenerOrdenesCompletasConSplitQuery()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .AsSplitQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Línea 1: public List ObtenerOrdenesCompletasConSplitQuery() → declara el método.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.AsNoTrackingWithIdentityResolution()` → aplica resolución de identidad.

Línea 5: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 6: `.Include(o => o.Detalle)` → incluye la referencia al detalle.

Línea 7: `.Include(o => o.OrdenesAleaciones)` → incluye la colección de entidades intermedias.

Línea 8: `.ThenInclude(oa => oa.Aleacion)` → incluye la aleación de cada entidad intermedia.

Línea 9: `.AsSplitQuery()` → divide la consulta en varias.

Línea 10: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 11: `.ToList();` → materializa la consulta.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarCompletaConSplitQuery()
{
    Console.WriteLine("\n--- Completa con AsSplitQuery ---");

    var cronometro = Stopwatch.StartNew();
    var ordenes = _unidad.Ordenes.ObtenerOrdenesCompletasConSplitQuery();
    cronometro.Stop();

    Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    foreach (var orden in ordenes)
    {
        var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica;
        Console.WriteLine($"  {orden.NumeroOrden}: {orden.Planchas.Count} planchas |
{orden.OrdenesAleaciones.Count} aleaciones | Detalle: {detalle}");
    }
}
```

Línea 1: `private void DemostrarCompletaConSplitQuery()` → declara el método.

Línea 3: `Console.WriteLine("\n--- Completa con AsSplitQuery ---");` → muestra la cabecera.

Línea 5: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 6: `var ordenes = _unidad.Ordenes.ObtenerOrdenesCompletasConSplitQuery();` → carga las órdenes.

Línea 7: `cronometro.Stop();` → detiene el cronómetro.

Línea 9: `Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el número de órdenes y el tiempo.

Línea 10: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 12: `var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica;` → obtiene el detalle.

Línea 13: `Console.WriteLine($" {orden.NumeroOrden}: {orden.Planchas.Count} planchas | {orden.OrdenesAleaciones.Count} aleaciones | Detalle: {detalle}");` → muestra los datos.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarCompletaConSplitQuery();
```

| Paso 5: Ejecutar dotnet run y comparar el tiempo.

Resultado esperado: el método carga todas las entidades relacionadas en varias consultas separadas sin producto cartesiano. El tiempo es mayor que con Include simple porque se ejecutan más consultas, pero el número de filas transferidas es menor.

| Analogía final

Las soluciones al problema N+1 en una acería son como las distintas formas de pedir las carpetas al archivo central. Include es como pedir en un solo viaje la carpeta de la orden y todas sus carpetas de planchas. ThenInclude es como pedir también las carpetas de aleaciones que están dentro de las carpetas de planchas. Las proyecciones son como pedir solo los datos que se necesitan de cada carpeta, sin traer la carpeta completa. AsSplitQuery es como pedir en varios viajes separados las carpetas de órdenes, las de planchas y las de aleaciones, evitando que el archivo tenga que combinar todo en un solo montón. Cada técnica tiene su coste y su beneficio. Include es rápido cuando solo hay una colección. Las proyecciones son más eficientes cuando solo se necesitan algunos datos. AsSplitQuery es más eficiente cuando hay varias colecciones. La combinación de técnicas permite resolver el problema N+1 de forma óptima según el escenario.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de solución al problema N+1 a la interfaz IOrdenRepositorio.

Implementado los métodos en OrdenRepositorio.

Creado el caso de uso SolucionN1UseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con órdenes, planchas, detalle y aleaciones.

Ejecutado las demostraciones de Include, ThenInclude, AsSplitQuery y proyección.

Comparado el tiempo de cada solución.

Analizado el SQL generado por cada técnica.

Diagnosticado el error de eliminar AsSplitQuery.

Creado el método ObtenerOrdenesCompletasConSplitQuery.
